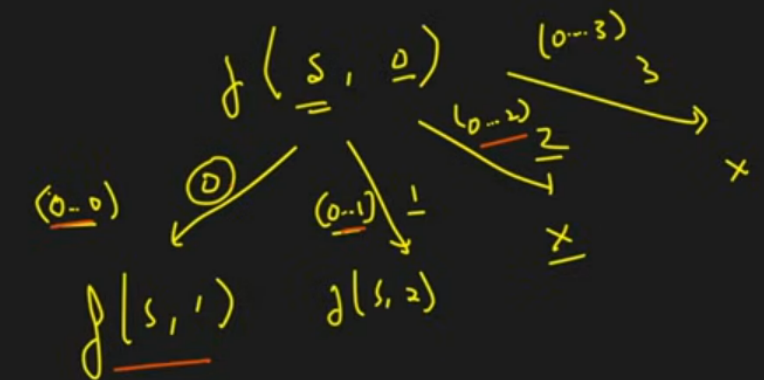


2.80
 {a, a, b, b}
 {a, a, bb}
 {a, b, b}
 {a, bb}

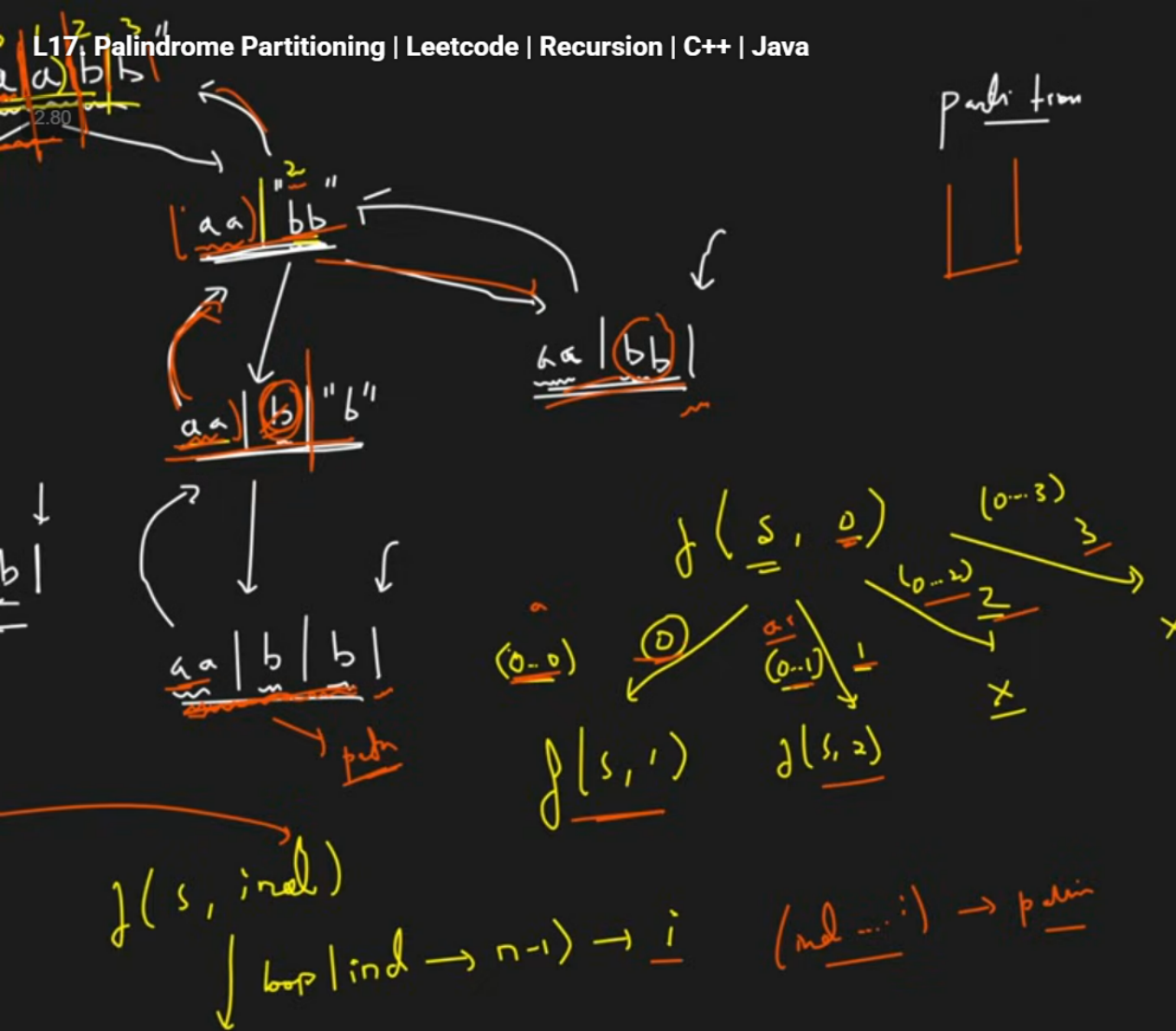


partition



$f(s, i, \text{end})$
 loop $i \rightarrow n-1 \rightarrow i$
 (end...i) $\rightarrow$ partition

L17: Palindrome Partitioning | LeetCode | Recursion | C++ | Java



```

class Solution {
public:
    vector<vector<string>> partition(string s) {
        vector<vector<string>> > res;
        vector<string> path;
        func(0, s, path, res);
        return res;
    }

    void func(int index, string s, vector<string> &path,
              vector<vector<string>> &res) {
        if(index == s.size()) {
            res.push_back(path);
            return;
        }
        for(int i = index; i < s.size(); ++i) {
            if(isPalindrome(s, index, i)) {
                path.push_back(s.substr(index, i - index + 1));
                func(i+1, s, path, res);
                path.pop_back();
            }
        }
    }

    bool isPalindrome(string s, int start, int end) {
        while(start <= end) {
            if(s[start++] != s[end--])
                return false;
        }
        return true;
    }
};

```

ChatGPT

New chat

Library

Projects

Scheduled

Plugins

Codex

More

Pinned

LeetCode Problem Solving Guide

SQL Hints And Guidance

Recents

Check Palindrome Code

C++ Break Continue Loops

Draw Subsequence Dry Run

Full Subsets Code

Create Solved Jupyter Notebook

Ladder Matrix Cpp Solution

MCQ And Coding Answers

Create Apple OA Roadmap

Assess First Run Percentile

Authorization answer

C++ solution request

Harsh Tanwar

Visualize it

```
s = a a b
  ↑ ↑
idx i
```

We want everything from `idx` through `i`.

Therefore:

```
number of characters
=
end - start + 1

=
i - idx + 1
```

That's where this comes from:

```
// C++
string t = s.substr(idx, i - idx + 1);
```

One more example

Suppose:

```
s = "abcde"
   01234

idx = 1
i   = 3
```

We want:

+ Ask anything

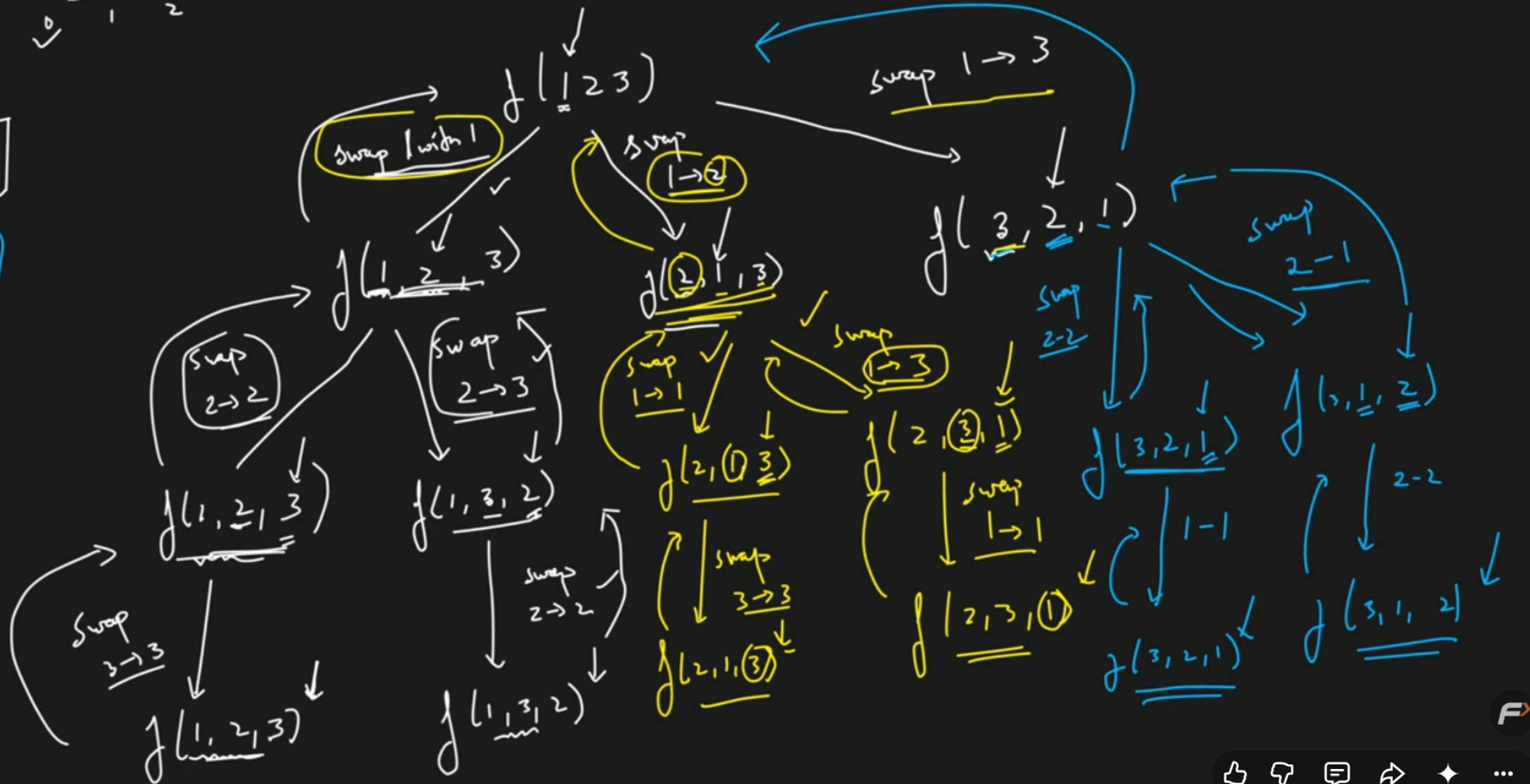
Think

L13. Print all Permutations of a String/Array | Recursion | Approach - 2

2.30

$[1, 2, 3]$ $n=3$
 $\downarrow$ 1 2

1 2 3
 1 3 2
 2 1 3
 2 3 1
 3 2 1
 3 1 2



$[1, 2, 3]$ $n=3$

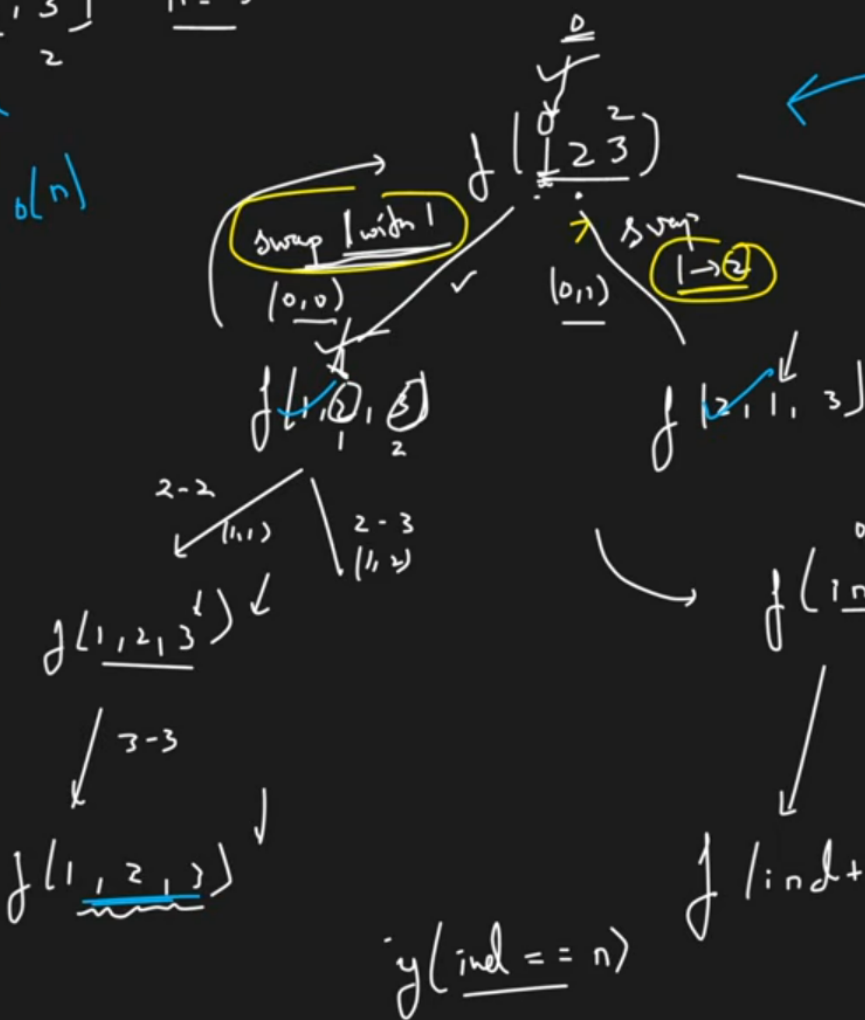
ds,

1	2	3
1	3	2
2	1	3
2	3	1
3	2	1
3	1	2

ans.

$O(n!)$

$O(n)$



swap 1 → 3
(0, 2)

2, 1

swap

$f(ind, arr)$

$i \rightarrow (ind \rightarrow n-1)$
swap($a[ind]$, $a[i]$)

$f(ind+1, arr)$

$if(ind == n)$

TC $\rightarrow n! \times (n)$
SC $\rightarrow$

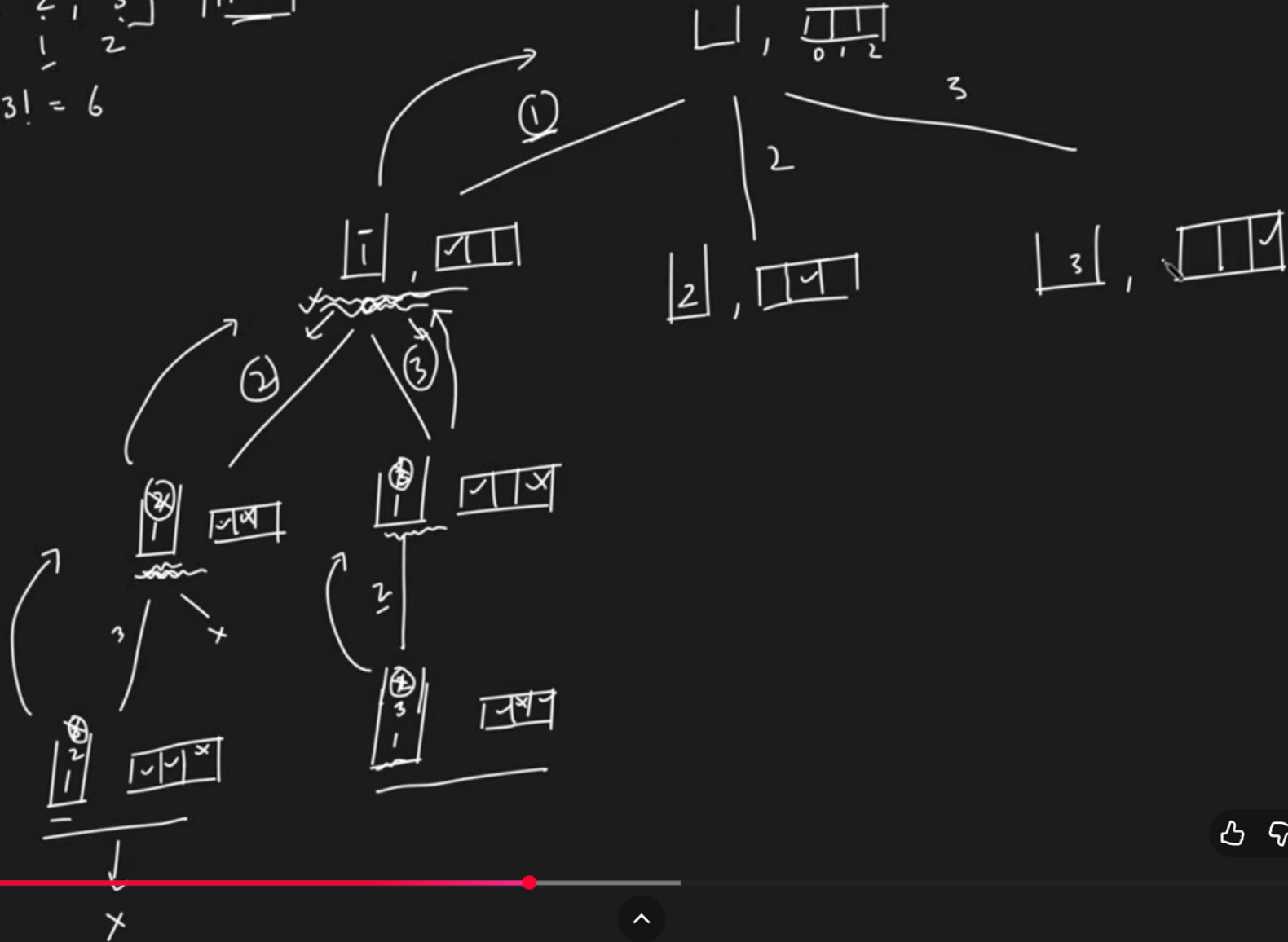
L12. Print all Permutations of a String/Array | Recursion | Approach - 1

2.00

$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \end{bmatrix}$ | $n = 3$

$$n! = 3! = 6$$

$\begin{bmatrix} 1, 2, 3 \\ 1, 3, 2 \end{bmatrix}$

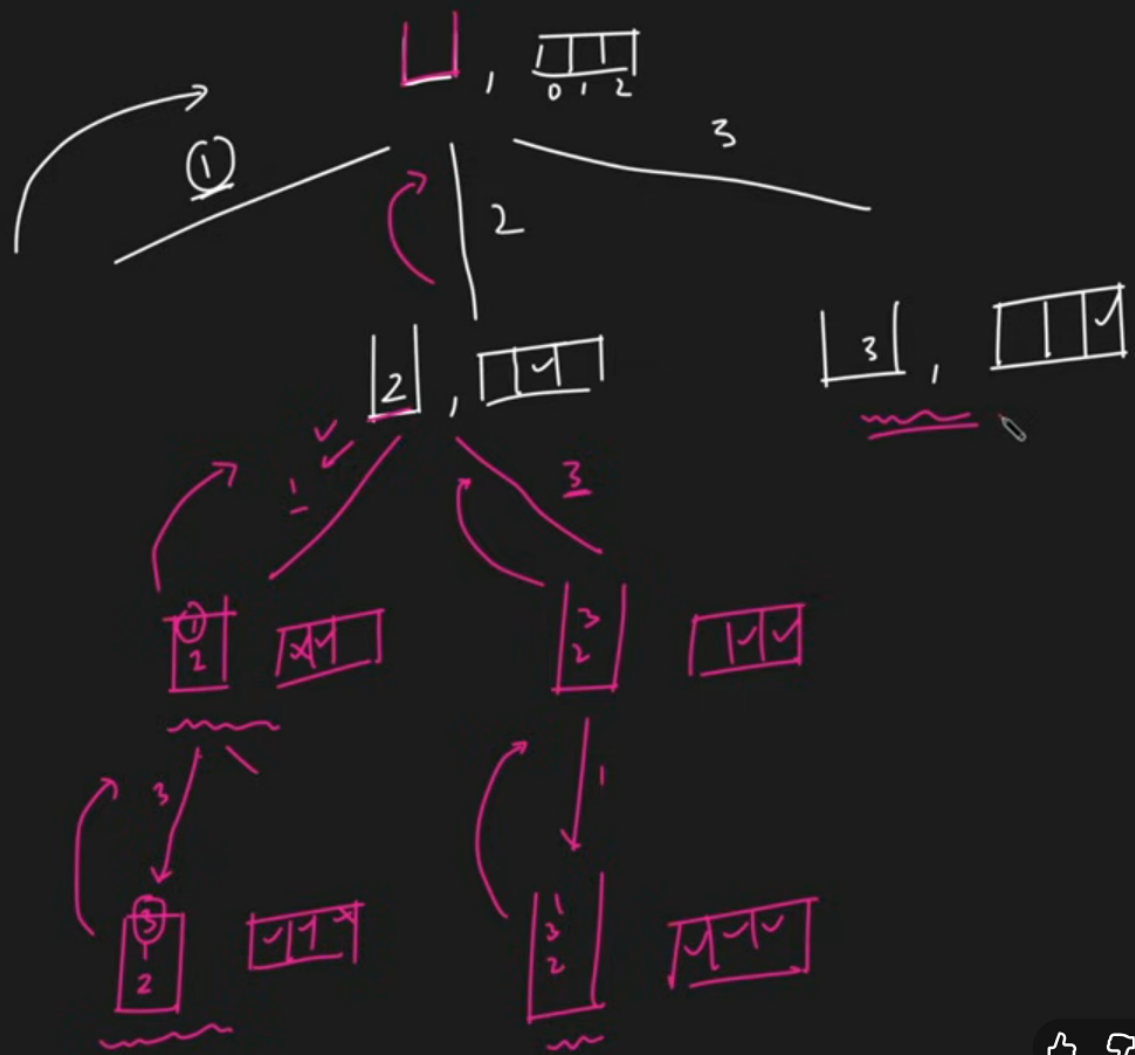


L12. Print all Permutations of a String/Array | Recursion | Approach - 1

2.00

$\begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \end{bmatrix}$ $n=3$
 $n! = 3! = 6$

$\begin{bmatrix} 1, 2, 3 \\ 1, 3, 2 \\ 2, 1, 3 \\ 2, 3, 1 \end{bmatrix}$



L12. Print all Permutations of a String/Array | Recursion | Approach - 1

2.00

$$n! = 3! = 6$$

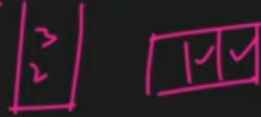
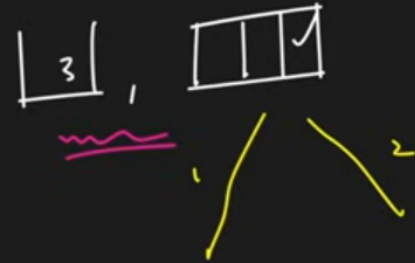
$f(ds, map)$

$\left\{ \begin{array}{l} ds.add(a[i]) \\ map[i] = 1 \end{array} \right\}$ loop $(0 - n-1)$
if $(i \text{ is } !map)$

$f(ds, map)$

Ans.

$(ds.size == n)$



$\left[\begin{array}{l} \underline{TC} \rightarrow n! \times n. \\ \underline{SC} \rightarrow o(n) + o(n) \end{array} \right]$




```
1 class Solution {
2 private:
3     void recurPermute(vector<int> &ds, vector<int> &nums, vector<vector<int>> &ans, int freq[]) {
4         if(ds.size() == nums.size()) {
5             ans.push_back(ds);
6             return;
7         }
8         for(int i = 0; i < nums.size(); i++) {
9             if(!freq[i]) {
10                 ds.push_back(nums[i]);
11                 freq[i] = 1;
12                 recurPermute(ds, nums, ans, freq);
13                 freq[i] = 0;
14                 ds.pop_back();
15             }
16         }
17     }
18 public:
19     vector<vector<int>> permute(vector<int> & nums) {
20         vector<vector<int>> ans;
21         vector<int> ds;
22         int freq[nums.size()] = {0};
23         for(int i = 0; i < nums.size(); i++) freq[i] = 0;
24         recurPermute(ds, nums, ans, freq);
25         return ans;
26     }
27 }
```

